

SQL Complete Notes

Deep Notes in English + Hinglish

For @raw_data_devi SQL Series & Data Analyst Prep

Chapter 1: Introduction to SQL

1.1 What is SQL?

English: SQL (Structured Query Language) is a special language used to communicate with databases. Just like we use our everyday language to talk to other people, SQL is the language we use to ask a database for data, update it, or delete it.

Hinglish: SQL (Structured Query Language) ek special language hai jo hum databases ke saath baat karne ke liye use karte hain. Jaise humari normal language hai jisse hum ek dusre se baat karte hain, waise hi SQL wo language hai jisse hum database se data maangte hain, usse update karte hain, ya delete karte hain.

Note: RDBMS ka matlab hai Relational Database Management System — jaise MySQL, PostgreSQL, SQL Server, Oracle.

1.2 Why Learn SQL?

English: Today, data is everywhere — companies, apps, banks, e-commerce platforms, all rely on it. And almost all of this data lives inside a database. If you want to view that data, understand it, or pull out insights from it, knowing SQL is essential.

Hinglish: Aaj ke time mein data har jagah hai — companies, apps, banks, e-commerce sab jagah. Aur ye sara data database mein store hota hai. Agar tumhe us data ko dekhna hai, samajhna hai, ya usse insights nikalne hain, toh SQL aana zaroori hai.

- Data Analyst / Data Scientist banne ke liye SQL sabse important skill hai.
- Almost har company (chhoti ho ya badi) apna data relational database mein store karti hai.
- SQL seekhna easy hai kyunki iski syntax English jaisi hi lagti hai (SELECT, WHERE, ORDER BY).

1.3 SQL vs Database vs DBMS

Term	Meaning
Database	Organized collection of data (jaise ek Excel file with many sheets)
DBMS	Software jo database ko manage karta hai (e.g., MySQL Server)
SQL	Language jisse hum DBMS ko instructions dete hain

1.4 Types of SQL Commands

English: SQL commands are divided into 5 major categories. Understanding this is important because it is a common interview question.

Hinglish: SQL commands ko 5 major categories mein baanta gaya hai. Ye samajhna bohot zaroori hai kyunki interviews mein bhi ye poocha jaata hai.

Category	Full Form	Commands	Kaam
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Table/database ka structure banana ya badalna
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Data ko add/change/remove karna
DQL	Data Query Language	SELECT	Data ko read/fetch karna
DCL	Data Control Language	GRANT, REVOKE	Permissions dena/lena
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Transactions manage karna

1.5 Quick Revision

- SQL = language to talk to databases.
- RDBMS store karta hai data ko tables ke form mein (rows aur columns).
- 5 types of commands: DDL, DML, DQL, DCL, TCL.

Chapter 2: Environment Setup (MySQL Workbench)

2.1 Installing MySQL & MySQL Workbench

English: MySQL is a popular open-source RDBMS. MySQL Workbench is a GUI tool that lets us write and run queries easily — the same tool used throughout this SQL series.

Hinglish: MySQL ek popular open-source RDBMS hai. MySQL Workbench ek GUI tool hai jisse hum queries easily likh aur run kar sakte hain — jaise tum apne SQL series mein use karti ho.

- MySQL Community Server install karo (database engine).
- MySQL Workbench install karo (visual interface — query likhne ke liye).
- Local connection banao (usually root user + password se).

2.2 Creating a Database

English: First, let's create a database — here is the store_db example used throughout these notes.

Hinglish: Sabse pehle ek database banate hain, jaise tumhare store_db ka example hai.

```
CREATE DATABASE store_db;  
USE store_db;
```

Note: USE statement bataata hai ki ab jitni bhi queries chalengi, wo store_db ke andar chalengi.

2.3 MySQL Workbench Interface Basics

- Query Tab — yahan tum apni SQL queries type karti ho.
- Schemas Panel (left side) — sare databases aur unke tables dikhte hain.
- Lightning bolt icon — query run karne ke liye (Ctrl+Enter bhi kaam karta hai).
- Result Grid — query ka output yahin table format mein dikhta hai.

2.4 Common Mistakes

- ⚠ Semicolon (;) bhool jaana — MySQL mein har statement ke end mein semicolon zaroori hai.
- ⚠ USE database bhoolna aur galat database mein query chala dena.
- ✓ Query run karne se pehle hamesha check karo kaunsa schema highlighted/selected hai (bold text schemas panel mein).

Chapter 3: SQL Data Types

English: Before creating a table, it's important to know what type of data each column will store. This is the same concept as int, float, and str in Python — just with slightly different names.

Hinglish: Table banane se pehle ye jaanna zaroori hai ki har column mein kis type ka data store hoga. Ye Python ke int, float, str jaisa hi concept hai, bas naam thode different hain.

3.1 Numeric Data Types

Type	Description	Example
INT	Whole number	25, 1000, -5
DECIMAL(p,s)	Exact decimal number (money ke liye best)	DECIMAL(10,2) → 1999.99
FLOAT / DOUBLE	Approximate decimal number	3.14159

3.2 String Data Types

Type	Description
CHAR(n)	Fixed-length string, hamesha n characters space leta hai
VARCHAR(n)	Variable-length string, max n characters (sabse common)
TEXT	Bada text data, jaise description ya comments

Note: Beginners ke liye rule: agar length fix hai (jaise state code 'MH') toh CHAR, warna VARCHAR use karo — 90% cases mein VARCHAR hi use hota hai.

3.3 Date and Time Data Types

Type	Format	Example
DATE	YYYY-MM-DD	2026-08-14
DATETIME	YYYY-MM-DD HH:MM:SS	2026-08-14 10:30:00
TIME	HH:MM:SS	14:30:00
YEAR	YYYY	2026

3.4 Boolean-like Type

English: MySQL does not have a separate BOOLEAN type — internally it is stored as TINYINT(1), where 0 means False and 1 means True.

Hinglish: MySQL mein alag se BOOLEAN type nahi hota — ye internally TINYINT(1) hota hai, jahan 0 = False aur 1 = True.

3.5 Quick Revision Table

- Numbers → INT, DECIMAL, FLOAT
- Text → VARCHAR (most common), CHAR, TEXT
- Dates → DATE, DATETIME, TIME

Chapter 4: Database Normalization

4.1 Why Normalization is Needed

English: Normalization is the process of organizing data into multiple related tables to reduce data duplication and avoid update/insert/delete anomalies. This is exactly why real databases split data into separate tables like customers and orders instead of one giant table.

Hinglish: Normalization ek process hai jisme data ko multiple related tables mein organize karte hain taaki data duplication kam ho aur update/insert/delete se related problems (anomalies) na aayein. Isi wajah se real databases mein data ko customers aur orders jaise alag tables mein todte hain, ek hi badi table mein nahi rakhte.

Note: Agar normalization na ho, toh ek hi customer ka naam har order ke saath baar-baar repeat hoga — jagah bhi waste hogi aur agar naam change karna ho toh har row update karni padegi.

4.2 First Normal Form (1NF)

English: A table is in 1NF when every column contains only atomic (single, indivisible) values, and there are no repeating groups or multi-valued columns.

Hinglish: Ek table 1NF mein tab hoti hai jab har column mein sirf atomic (single, indivisible) values hon, aur koi repeating groups ya multiple values ek column mein na ho.

```
-- Galat (1NF violation): ek column mein multiple phone numbers
customer_id | customer_name | phones
1           | Varsha       | '9876543210, 9123456789'

-- Sahi (1NF): alag rows ya alag table
customer_id | customer_name | phone
1           | Varsha       | 9876543210
1           | Varsha       | 9123456789
```

4.3 Second Normal Form (2NF)

English: A table is in 2NF when it is already in 1NF, and every non-key column depends on the entire primary key, not just part of it. This issue only comes up with composite (multi-column) primary keys.

Hinglish: Ek table 2NF mein tab hoti hai jab wo pehle se 1NF mein ho, aur har non-key column poore primary key par depend kare, uske sirf ek part par nahi. Ye issue tabhi aata hai jab primary key composite (multiple columns ka) ho.

Note: Example: agar table ka primary key (order_id, product_id) hai, lekin product_name sirf product_id par depend karta hai, toh ye 2NF violation hai — product_name ko alag products table mein rakhna chahiye.

4.4 Third Normal Form (3NF)

English: A table is in 3NF when it is already in 2NF, and no non-key column depends on another non-key column (no transitive dependency).

Hinglish: Ek table 3NF mein tab hoti hai jab wo pehle se 2NF mein ho, aur koi bhi non-key column kisi dusre non-key column par depend na kare (no transitive dependency).

-- Galat (3NF violation): city_pincode, city par depend karta hai, city_pincode par nahi
customer_id | customer_name | city | city_pincode

-- Sahi: city aur pincode ko alag table mein rakho
customers: customer_id, customer_name, city_id
cities: city_id, city_name, pincode

4.5 Quick Revision

- 1NF → har column mein sirf single/atomic value.
- 2NF → 1NF + poora primary key par non-key columns depend karein (composite keys ke case mein).
- 3NF → 2NF + koi transitive dependency na ho non-key columns ke beech.
- Real-world databases usually 3NF tak normalize karte hain — beyond that (BCNF, 4NF) rarely use hota hai.

Chapter 5: DDL — Creating & Modifying Tables

5.1 CREATE TABLE

English: A table is a structure that stores data in the form of rows and columns — very similar to an Excel sheet. The orders and customers tables were created in exactly this way.

Hinglish: Table ek structure hai jisme data rows aur columns ke form mein store hota hai — bilkul Excel sheet jaisa. Tumhare orders aur customers tables isi tarah bane the.

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  email VARCHAR(100),  
  city VARCHAR(50)  
);
```

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  amount DECIMAL(10,2),
```

```
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

5.2 ALTER TABLE

English: ALTER TABLE is used when you need to change the structure of an already existing table — such as adding a new column, removing an old one, or changing a data type.

Hinglish: ALTER TABLE ka use hota hai jab already bani hui table ka structure change karna ho — jaise naya column add karna, purana remove karna, ya data type badalna.

```
-- Naya column add karna
ALTER TABLE customers ADD COLUMN phone VARCHAR(15);

-- Column ka naam ya type change karna
ALTER TABLE customers MODIFY COLUMN phone VARCHAR(20);

-- Column delete karna
ALTER TABLE customers DROP COLUMN phone;
```

5.3 DROP vs TRUNCATE vs DELETE

English: These three commands often confuse beginners, so look carefully at this comparison table:

Hinglish: Ye teeno beginners ko confuse karte hain, isliye ye table dhyan se dekho:

Command	Kya delete karta hai	Structure bachta hai?	Rollback possible?
DROP TABLE	Poori table (data + structure)	Nahi	Nahi
TRUNCATE TABLE	Sirf data, structure rehta hai	Haan	Nahi (usually)
DELETE FROM table	Selected rows (ya sab rows)	Haan	Haan (with WHERE, transaction)

5.4 Constraints Overview

- PRIMARY KEY — har row ko uniquely identify karta hai, NULL allowed nahi.
- FOREIGN KEY — ek table ko dusri table se link karta hai.
- NOT NULL — column empty nahi ho sakta.
- UNIQUE — duplicate values allowed nahi.
- DEFAULT — agar value nahi di gayi toh ye default value use hogi.
- CHECK — condition define karta hai jo value ko satisfy karni hoti hai.

5.5 PRIMARY KEY vs FOREIGN KEY

English: These two keys are the backbone of table relationships, and interviewers love comparing them.

Hinglish: Ye dono keys table relationships ki neev hain, aur interviewers inhe compare karna bohot pasand karte hain.

PRIMARY KEY	FOREIGN KEY
Row ko uniquely identify karta hai apni hi table mein	Ek table ko dusri table se link karta hai
NULL allowed nahi	NULL allowed hai (agar match zaroori na ho)
Ek table mein sirf ek PRIMARY KEY ho sakti hai	Ek table mein multiple FOREIGN KEYS ho sakte hain
Automatically unique index bana deta hai	Referenced table ke PRIMARY KEY ko point karta hai

5.6 Common Mistakes

- ⚠️ **PRIMARY KEY** define kiye bina table banana — isse duplicate rows ka risk badh jaata hai.
- ⚠️ **DROP** aur **TRUNCATE** ko **DELETE** samajh lena — **DROP** se poori table hi gayab ho jaati hai!

Chapter 6: DML — Inserting & Modifying Data

6.1 INSERT INTO

English: Once a table is created, we use INSERT INTO to add data into it.

Hinglish: Table banane ke baad usme data daalne ke liye INSERT INTO use karte hain.

```
INSERT INTO customers (customer_id, customer_name, email, city)
VALUES (1, 'Varsha Sharma', 'varsha@email.com', 'Delhi');
```

Note: Multiple rows ek saath insert karne ke liye comma se separate karke values de sakte ho.

```
INSERT INTO customers (customer_id, customer_name, city) VALUES
(2, 'Rahul Verma', 'Mumbai'),
(3, 'Aisha Khan', 'Pune');
```

6.2 UPDATE

English: UPDATE is used to change data that already exists in a table.

Hinglish: Already existing data ko change karne ke liye UPDATE use hota hai.

```
UPDATE customers
SET city = 'Bengaluru'
WHERE customer_id = 2;
```

⚠️ **WHERE** clause bhool jaana — agar **WHERE** nahi likha toh saari table ki rows update ho jaayengi!

6.3 DELETE

```
DELETE FROM customers
WHERE customer_id = 3;
```

Note: DELETE FROM table_name (bina WHERE ke) saari rows delete kar dega, lekin table structure bacha rahega.

6.4 Quick Revision

- INSERT → naya data add karna
- UPDATE → existing data change karna (WHERE ke saath)
- DELETE → rows remove karna (WHERE ke saath)

Chapter 7: SELECT & Filtering Data

7.1 Basic SELECT Statement

English: SELECT is the most frequently used SQL command — it is how we 'fetch' (read) data from a table.

Hinglish: SELECT SQL ka sabse zyada use hone wala command hai — isse hum table se data 'fetch' (read) karte hain.

```
SELECT * FROM customers;

-- Specific columns select karna
SELECT customer_name, city FROM customers;
```

*Note: * ka matlab hai 'saare columns'. Production mein specific columns select karna best practice hai, kyunki * performance ko slow kar sakta hai large tables mein.*

7.2 WHERE Clause — Filtering Rows

English: The WHERE clause filters rows based on a condition — very similar to a filter in Excel.

Hinglish: WHERE clause condition ke basis par rows filter karta hai — bilkul Excel ke filter jaisa.

```
SELECT * FROM orders
WHERE amount > 5000;
```

7.3 Comparison Operators

Operator	Meaning
=	Equal to
!= or <>	Not equal to
>, <, >=, <=	Greater/less than (equal to)

7.4 Logical Operators — AND, OR, NOT

```
SELECT * FROM orders
WHERE amount > 1000 AND order_date >= '2026-01-01';

SELECT * FROM customers
WHERE city = 'Delhi' OR city = 'Mumbai';
```

7.5 BETWEEN, IN, LIKE, IS NULL

```
-- BETWEEN: range check karta hai (dono values inclusive)
SELECT * FROM orders WHERE amount BETWEEN 1000 AND 5000;

-- IN: multiple specific values check karta hai
SELECT * FROM customers WHERE city IN ('Delhi', 'Mumbai', 'Pune');

-- LIKE: pattern matching ke liye (% = kuch bhi characters, _ = ek character)
SELECT * FROM customers WHERE customer_name LIKE 'A%';

-- IS NULL / IS NOT NULL: empty values check karne ke liye
SELECT * FROM customers WHERE email IS NULL;
```

⚠ email = NULL likhna — ye kaam nahi karega! NULL check karne ke liye hamesha IS NULL / IS NOT NULL use karo, = ya != nahi.

7.6 DISTINCT — Removing Duplicates

```
SELECT DISTINCT city FROM customers;
```

7.7 Quick Revision

- SELECT columns FROM table WHERE condition — ye basic structure yaad rakho.
- AND/OR se multiple conditions combine hoti hain.
- NULL check karne ke liye IS NULL use karo.

Chapter 8: Sorting & Limiting Results

8.1 ORDER BY

English: ORDER BY lets us sort the result in ascending (smallest to largest) or descending (largest to smallest) order.

Hinglish: ORDER BY se hum result ko ascending (chhote se bade) ya descending (bade se chhote) order mein sort karte hain.

```
SELECT * FROM orders
ORDER BY amount DESC;

-- Default ASC hota hai
SELECT * FROM customers
ORDER BY customer_name ASC;
```

8.2 Sorting by Multiple Columns

```
SELECT * FROM orders
ORDER BY customer_id ASC, amount DESC;
```

English: Here, the result is first sorted by customer_id, and among rows with the same customer_id, it is further sorted by amount in descending order.

Hinglish: Yahan pehle customer_id ke basis par sort hoga, aur jinka customer_id same hai unke beech amount ke basis par (descending) sort hoga.

8.3 LIMIT — Restricting Number of Rows

English: LIMIT lets us fetch only the top N rows — for example, pulling out the 'Top 5 highest orders'.

Hinglish: LIMIT se hum sirf top N rows fetch karte hain — jaise 'Top 5 highest orders' nikalna.

```
SELECT * FROM orders
ORDER BY amount DESC
LIMIT 5;
```

Note: LIMIT ke saath OFFSET bhi use hota hai pagination ke liye — jaise LIMIT 5 OFFSET 10 ka matlab hai 11th se 15th row tak.

8.4 Quick Revision

- ORDER BY column ASC/DESC — sorting ke liye
- LIMIT n — sirf n rows dikhane ke liye
- In dono ka combo top-N queries (Top 10 customers, etc.) banane mein use hota hai — jo interview mein bohot poocha jaata hai.

Chapter 9: Aggregate Functions & Grouping

9.1 What are Aggregate Functions?

English: Aggregate functions take data from an entire column (or group) and return a single summary value — such as a total, average, or count.

Hinglish: Aggregate functions poore column (ya group) ka data lekar ek single summary value return karte hain — jaise total, average, count.

Function	Kaam
COUNT()	Rows ki ginti karta hai
SUM()	Values ka total nikalta hai
AVG()	Average (mean) nikalta hai
MIN()	Sabse chhota value
MAX()	Sabse bada value

```
SELECT COUNT(*) AS total_orders, SUM(amount) AS total_sales, AVG(amount) AS avg_order_value
FROM orders;
```

9.2 GROUP BY

English: GROUP BY lets us split data into categories and get a summary for each category — for example, 'the total order amount per customer'.

Hinglish: GROUP BY se hum data ko categories mein baant kar har category ka summary nikalte hain — jaise 'har customer ka total order amount'.

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

Note: Rule yaad rakho: SELECT mein jo bhi non-aggregate column ho, wo GROUP BY mein bhi hona chahiye.

9.3 HAVING — Filtering Groups

English: WHERE performs row-level filtering, but for group-level filtering we use HAVING (for example, showing only customers whose total spend is more than 10000).

Hinglish: WHERE row-level filtering karta hai, lekin group-level filtering ke liye HAVING use hota hai (jaise sirf wo customers dikhao jinka total spend 10000 se zyada hai).

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

9.4 WHERE vs HAVING

WHERE	HAVING
Rows filter karta hai (GROUP BY se pehle)	Groups filter karta hai (GROUP BY ke baad)
Aggregate function use nahi kar sakta	Aggregate function use kar sakta hai

9.5 Common Mistakes

⚠ WHERE mein aggregate function use karna, jaise WHERE SUM(amount) > 1000 — ye error dega.

⚠ **GROUP BY** mein wo columns na dena jo **SELECT** mein hain — MySQL kabhi allow kar deta hai but result galat aa sakta hai.

Chapter 10: Joins

10.1 Why Joins are Needed

English: In real-world databases, data is spread across multiple tables (because of normalization) — for example, customers in one table and orders in another. Joins are used to connect these tables based on their relationship.

Hinglish: Real-world databases mein data multiple tables mein spread hota hai (normalization ki wajah se) — jaise customers alag table mein aur orders alag table mein. Joins ka use in tables ko relationship ke basis par connect karne ke liye hota hai.

10.2 INNER JOIN

English: INNER JOIN returns only the rows that have a match in both tables.

Hinglish: INNER JOIN sirf wahi rows return karta hai jinka match dono tables mein ho.

```
SELECT c.customer_name, o.order_id, o.amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

10.3 LEFT JOIN (LEFT OUTER JOIN)

English: LEFT JOIN returns all rows from the left table (customers), whether or not there is a match in the right table (orders). When there is no match, NULL appears.

Hinglish: LEFT JOIN left table (customers) ki saari rows deta hai, chahe right table (orders) mein match ho ya na ho. Match na hone par NULL aata hai.

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Note: Ye query use hoti hai jab tumhe pata karna ho 'kaunse customers ne abhi tak koi order nahi kiya' — WHERE o.order_id IS NULL laga do.

10.4 RIGHT JOIN (RIGHT OUTER JOIN)

English: RIGHT JOIN is the opposite of LEFT JOIN — all rows from the right table are returned, and NULL appears where there is no match in the left table.

Hinglish: RIGHT JOIN LEFT JOIN ka opposite hai — right table ki saari rows aati hain, left table mein match na ho toh NULL.

```
SELECT c.customer_name, o.order_id
FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

10.5 FULL OUTER JOIN

English: It returns all rows from both tables (matched or not). MySQL does not support FULL OUTER JOIN directly — it is simulated by combining a LEFT JOIN and a RIGHT JOIN with UNION.

Hinglish: Dono tables ki saari rows deta hai (match ho ya na ho). MySQL mein directly FULL OUTER JOIN support nahi hota — LEFT JOIN aur RIGHT JOIN ko UNION se combine karke banate hain.

```
SELECT c.customer_name, o.order_id FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
UNION
SELECT c.customer_name, o.order_id FROM customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;
```

10.6 SELF JOIN

English: Sometimes a table needs to be joined with itself — for example, in an employees table, a manager is also an employee.

Hinglish: Kabhi kabhi ek table ko khud ke saath join karna padta hai — jaise employees table mein manager bhi ek employee hi hota hai.

```
SELECT e.employee_name, m.employee_name AS manager_name
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id;
```

10.7 CROSS JOIN

English: It combines every row of one table with every row of the other table (a Cartesian product). It is rarely used in practice, but understanding the concept is still important.

Hinglish: Har row ko dusri table ki har row ke saath combine karta hai (Cartesian product). Rarely use hota hai, but concept jaanna important hai.

10.8 Joins Visual Summary

Join Type	Result
INNER JOIN	Sirf matching rows
LEFT JOIN	Left table ki saari + matching right rows
RIGHT JOIN	Right table ki saari + matching left rows
FULL OUTER JOIN	Dono tables ki saari rows

10.9 Common Mistakes

⚠️ ON condition galat likhna ya bhool jaana — isse Cartesian product ban jaata hai (bohot saari duplicate rows).

⚠️ Table alias use na karna jab dono tables mein same column name ho — 'ambiguous column' error aata hai.

Chapter 11: Subqueries

11.1 What is a Subquery?

English: A subquery is a query written inside another query. The inner query executes first, and the outer query uses its result.

Hinglish: Subquery ek query ke andar dusri query hoti hai. Inner query pehle execute hoti hai, uska result outer query use karti hai.

11.2 Subquery in WHERE Clause

```
SELECT customer_name FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders WHERE amount > 5000
);
```

11.3 Subquery in FROM Clause (Derived Table)

```
SELECT city, AVG(total_spent) AS avg_spent_per_city
FROM (
    SELECT c.city, SUM(o.amount) AS total_spent
    FROM customers c JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.city, c.customer_id
) AS city_totals
GROUP BY city;
```

11.4 Correlated Subquery

English: A correlated subquery re-executes for every row of the outer query — which is why it can be slower than a regular subquery.

Hinglish: Correlated subquery outer query ke har row ke liye dobara execute hoti hai — isliye ye normal subquery se slow ho sakti hai.

```
SELECT customer_name FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

11.5 Subquery vs JOIN

Note: Jab bhi possible ho, JOIN prefer karo — usually JOIN performance mein better hota hai subquery ke comparison mein, especially large datasets par.

Chapter 12: Common Table Expressions (CTEs)

12.1 What is a CTE?

English: A Common Table Expression (CTE) is a temporary named result set defined using the WITH keyword, which you can reference inside a SELECT, INSERT, UPDATE, or DELETE statement. It exists only for the duration of that single query.

Hinglish: Common Table Expression (CTE) ek temporary named result set hota hai jo WITH keyword se define karte hain, jisse hum SELECT, INSERT, UPDATE, ya DELETE statement ke andar reference kar sakte hain. Ye sirf usi query ke duration tak exist karta hai.

Note: CTE ek subquery jaisa hi kaam karta hai, lekin isse likhna aur padhna dono easy hote hain — especially jab query complex ho jaaye.

12.2 Basic CTE Syntax (WITH clause)

```
WITH high_value_customers AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders
    GROUP BY customer_id
```

```

HAVING SUM(amount) > 10000
)
SELECT c.customer_name, h.total_spent
FROM high_value_customers h
JOIN customers c ON c.customer_id = h.customer_id;

```

12.3 CTE vs Subquery

English: CTEs and subqueries can often achieve the same result, but CTEs are more readable, can be reused multiple times in the same query, and support recursion — which subqueries cannot do.

Hinglish: CTEs aur subqueries kaafi baar same result de sakte hain, lekin CTEs zyada readable hote hain, same query mein multiple baar reuse ho sakte hain, aur recursion support karte hain — jo subqueries nahi kar sakti.

CTE (WITH)	Subquery
Readable, top se likha jaata hai	Query ke andar nested hota hai, padhna mushkil ho sakta hai
Same query mein multiple baar reuse ho sakta hai	Har baar dobara likhna padta hai
Recursive queries support karta hai	Recursion support nahi karta

12.4 Recursive CTEs

English: A recursive CTE refers to itself, which is useful for hierarchical data like an organization chart (employee-manager relationships).

Hinglish: Recursive CTE khud ko refer karta hai, jo hierarchical data ke liye useful hota hai — jaise organization chart (employee-manager relationship).

```

WITH RECURSIVE employee_hierarchy AS (
    SELECT employee_id, employee_name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, e.employee_name, e.manager_id, eh.level + 1
    FROM employees e
    JOIN employee_hierarchy eh ON e.manager_id = eh.employee_id
)
SELECT * FROM employee_hierarchy;

```

12.5 Multiple CTEs in One Query

English: You can define more than one CTE in the same query by separating them with commas — each one can even reference the CTEs defined before it.

Hinglish: Ek hi query mein multiple CTEs comma se separate karke define kar sakte hain — har CTE apne se pehle wali CTEs ko bhi reference kar sakta hai.

```

WITH monthly_sales AS (
    SELECT MONTH(order_date) AS month, SUM(amount) AS total
    FROM orders GROUP BY MONTH(order_date)
),
best_month AS (
    SELECT month FROM monthly_sales ORDER BY total DESC LIMIT 1
)
SELECT * FROM best_month;

```

Chapter 13: Set Operations

13.1 UNION and UNION ALL

English: UNION combines the results of two or more SELECT queries. UNION removes duplicate rows, while UNION ALL keeps them.

Hinglish: UNION do ya zyada SELECT queries ke results ko combine karta hai. UNION duplicates hata deta hai, UNION ALL nahi.

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

Note: UNION karne ke liye dono queries mein columns ki number aur data type match hone chahiye.

13.2 INTERSECT and EXCEPT

English: Recent versions of MySQL (8.0.31+) support INTERSECT and EXCEPT natively. In older versions, they are simulated using JOINS or subqueries.

Hinglish: MySQL ke recent versions (8.0.31+) mein INTERSECT aur EXCEPT support hota hai. Purane versions mein inhe JOIN ya subquery se simulate karte hain.

Chapter 14: String, Date & Numeric Functions

14.1 String Functions

Function	Kaam	Example
UPPER() / LOWER()	Case badalna	UPPER('varsha') → VARSHA
LENGTH()	String ki length	LENGTH('sql') → 3
CONCAT()	Strings jodna	CONCAT('Hi ', 'Varsha')
TRIM()	Extra spaces hatana	TRIM(' hi ') → 'hi'
SUBSTRING()	Part of string nikalna	SUBSTRING('Hello',1,3) → 'Hel'
REPLACE()	Text replace karna	REPLACE('abc','a','x') → 'xbc'

14.2 Date Functions

Function	Kaam
NOW()	Current date aur time
CURDATE()	Current date
DATE_ADD() / DATE_SUB()	Date mein din/mahine add/subtract karna
DATEDIFF()	Do dates ke beech din ka difference
YEAR() / MONTH() / DAY()	Date se saal/mahina/din nikalna

```
SELECT order_id, order_date,
DATEDIFF(NOW(), order_date) AS days_since_order
FROM orders;
```

14.3 Numeric Functions

Function	Kaam
ROUND()	Number ko round karna
CEIL() / FLOOR()	Upar/neeche ki taraf round karna
ABS()	Absolute (positive) value

MOD()	Remainder nikalna
-------	-------------------

14.4 NULL Handling Functions

English: NULL values need special functions to handle safely, since normal comparisons don't work on NULL.

Hinglish: NULL values ko safely handle karne ke liye special functions chahiye hote hain, kyunki normal comparisons NULL par kaam nahi karte.

Function	Kaam
COALESCE(a, b, c...)	Pehli non-NULL value return karta hai list mein se
IFNULL(a, default)	Agar a NULL hai toh default value deta hai (MySQL specific)
NULLIF(a, b)	Agar a = b hai toh NULL return karta hai, warna a

```
SELECT customer_name, IFNULL(email, 'Not Provided') AS email
FROM customers;
```

```
SELECT COALESCE(phone, email, 'No Contact Info') AS contact
FROM customers;
```

Note: COALESCE zyada flexible hai kyunki ye multiple columns check kar sakta hai, jabki IFNULL sirf do values ke saath kaam karta hai.

14.5 GROUP_CONCAT()

English: GROUP_CONCAT combines values from multiple rows within a group into a single comma-separated string — very handy for summary reports.

Hinglish: GROUP_CONCAT ek group ke andar multiple rows ki values ko ek single comma-separated string mein combine kar deta hai — summary reports ke liye bohot useful.

```
SELECT customer_id, GROUP_CONCAT(order_id ORDER BY order_id SEPARATOR ', '
) AS all_orders
FROM orders
GROUP BY customer_id;
```

Note: Example output: customer_id = 1, all_orders = '101, 105, 110'

14.6 CASE Statement — Conditional Logic

English: CASE works like if-else in SQL — it returns a new value based on the value of a column.

Hinglish: CASE SQL ka if-else jaisa hota hai — column ki value ke basis par naya value return karta hai.

```
SELECT order_id, amount,
CASE
    WHEN amount >= 10000 THEN 'High Value'
    WHEN amount >= 5000 THEN 'Medium Value'
    ELSE 'Low Value'
END AS order_category
FROM orders;
```

Chapter 15: Views

15.1 What is a View?

English: A view is a 'virtual table' that is actually a saved version of a query. It does not physically store data — every time you access the view, the query inside it runs again.

Hinglish: View ek 'virtual table' hoti hai jo actually kisi query ka saved version hoti hai. Isme data physically store nahi hota — jab bhi view ko access karte ho, uske andar ki query fir se run hoti hai.

```
CREATE VIEW high_value_orders AS
SELECT * FROM orders WHERE amount > 10000;

-- Ab isse normal table ki tarah query kar sakte ho
SELECT * FROM high_value_orders;
```

15.2 Why Use Views?

- Complex queries ko simplify karke reuse karna.
- Sensitive columns ko hide karke sirf zaroori data dikhana.
- Reporting ke liye ek consistent 'clean' data source dena.

Chapter 16: Indexes

16.1 What is an Index?

English: An index is a data structure that speeds up how quickly data can be found in a table — just like the index page of a book, which takes you directly to a topic's page number without flipping through the whole book.

Hinglish: Index ek data structure hai jo table mein data dhoondhne ki speed badhata hai — bilkul kisi book ke index page jaisa, jahan se hum seedha topic ke page number tak pahunch jaate hain, poori book palte bina.

```
CREATE INDEX idx_customer_city ON customers(city);
```

Note: PRIMARY KEY automatically ek index create karta hai.

16.2 Trade-offs of Indexing

✓ SELECT queries fast ho jaati hain (especially WHERE, JOIN, ORDER BY columns par).

⚠ Har column par index laga dena — isse INSERT/UPDATE/DELETE slow ho jaate hain, kyunki har change par index bhi update hota hai.

Chapter 17: Window Functions

17.1 What are Window Functions?

English: Window functions look similar to GROUP BY, but GROUP BY collapses rows into one summary row per group, while a window function keeps every row as-is and adds an extra calculated column.

Hinglish: Window functions GROUP BY jaise lagte hain, lekin GROUP BY rows ko collapse (combine) kar deta hai, jabki window function har row ko as-it-is rakhte hue ek extra calculated column deta hai.

17.2 ROW_NUMBER(), RANK(), DENSE_RANK()

```
SELECT customer_id, amount,  
ROW_NUMBER() OVER (ORDER BY amount DESC) AS row_num,  
RANK() OVER (ORDER BY amount DESC) AS rank_num  
FROM orders;
```

Note: RANK() equal values ko same rank deta hai aur agla number skip karta hai. DENSE_RANK() equal values ko same rank deta hai lekin number skip nahi karta.

17.3 PARTITION BY

English: PARTITION BY splits data into groups (similar to GROUP BY), but the window function is applied within each group without collapsing the rows.

Hinglish: PARTITION BY data ko groups mein baant deta hai (GROUP BY jaisa), lekin har group ke andar window function apply hota hai, rows collapse nahi hoti.

```
SELECT customer_id, order_id, amount,  
RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS  
rank_within_customer  
FROM orders;
```

17.4 SUM(), AVG() as Window Functions

```
SELECT order_id, amount,  
SUM(amount) OVER (ORDER BY order_date) AS running_total  
FROM orders;
```

Chapter 18: Transactions (TCL)

18.1 What is a Transaction?

English: A transaction is a group of SQL statements that either all execute completely, or none of them execute at all (all-or-nothing) — this keeps data consistent. For example, in a bank transfer, deducting money from one account and adding it to another must both happen together.

Hinglish: Transaction ek group of SQL statements hai jo ya toh sab poori tarah execute hoti hain, ya ek bhi execute nahi hoti (all-or-nothing) — isse data consistency banti hai. Jaise bank transfer: paise ek account se katne aur dusre mein add hone, dono ek saath hone chahiye.

18.2 COMMIT, ROLLBACK, SAVEPOINT

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;  
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;
```

```
COMMIT; -- changes permanently save ho jaayengi  
-- ya agar kuch galat ho jaaye:  
-- ROLLBACK; -- saare changes undo ho jaayenge
```

18.3 ACID Properties

Property	Meaning
Atomicity	Sab ya kuch nahi — transaction पूरी या बिल्कुल नही
Consistency	Database ek valid state se dusre valid state mein jaata hai
Isolation	Ek transaction dusre transaction ko affect nahi karta
Durability	COMMIT ke baad changes permanent ho jaate hain, chahe system crash ho jaaye

Chapter 19: Stored Procedures & Triggers (Basics)

19.1 Stored Procedures

English: A stored procedure is a saved block of SQL statements that can be called again and again — useful for code reusability, similar to a function in Python.

Hinglish: Stored Procedure SQL statements ka ek saved block hota hai jise hum baar baar call kar sakte hain — code reusability ke liye useful, jaise Python mein function hota hai.

```
DELIMITER //
CREATE PROCEDURE GetHighValueOrders()
BEGIN
    SELECT * FROM orders WHERE amount > 10000;
END //
DELIMITER ;

CALL GetHighValueOrders();
```

19.2 Triggers

English: A trigger is an automatic action that runs when a specific event happens on a table (INSERT/UPDATE/DELETE).

Hinglish: Trigger ek automatic action hota hai jo tab chalta hai jab table par koi specific event hota hai (INSERT/UPDATE/DELETE).

```
CREATE TRIGGER before_order_insert
BEFORE INSERT ON orders
FOR EACH ROW
SET NEW.order_date = IFNULL(NEW.order_date, CURDATE());
```

Chapter 20: SQL Query Execution Order (Interview Favorite)

English: This is a very common interview topic — the order in which SQL is written is different from the order in which it actually executes.

Hinglish: Ye topic bohot common interview question hai — SQL likhne ka order aur execute hone ka order alag hota hai.

Written Order	Execution Order
1. SELECT	1. FROM
2. FROM	2. WHERE
3. WHERE	3. GROUP BY
4. GROUP BY	4. HAVING

5. HAVING	5. SELECT
6. ORDER BY	6. ORDER BY
7. LIMIT	7. LIMIT

Note: Isliye WHERE mein column alias (jo SELECT mein banaya) use nahi kar sakte — kyunki WHERE, SELECT se pehle execute hota hai. Lekin ORDER BY mein alias chalta hai, kyunki wo SELECT ke baad execute hota hai.

Chapter 21: SQL Interview Question Bank

English: These are some of the most commonly asked SQL interview questions. Practicing these will help both with interviews and with making strong revision content.

Hinglish: Ye kuch sabse zyada poochhe jaane wale SQL interview questions hain. Inhe practice karne se interviews mein bhi help milegi aur strong revision content banane mein bhi.

21.1 Find the Nth Highest Salary

```
-- Using LIMIT and OFFSET (Nth = 3rd highest example)
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;

-- Using DENSE_RANK (handles ties correctly)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 3;
```

Note: DENSE_RANK() approach better hai jab salary mein duplicate values ho sakti hain — LIMIT/OFFSET unhe alag-alag rank nahi deta.

21.2 Find Duplicate Rows

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

21.3 Delete Duplicate Rows (Keep One Copy)

```
DELETE c1 FROM customers c1
INNER JOIN customers c2
WHERE c1.customer_id > c2.customer_id
AND c1.email = c2.email;
```

Note: Ye query har duplicate group mein se sabse chhote customer_id wali row rakh ke baaki sab delete kar deti hai.

21.4 Employees Earning More Than Their Manager

```
SELECT e.employee_name
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

21.5 Running Total / Cumulative Sum

```
SELECT order_id, order_date, amount,  
SUM(amount) OVER (ORDER BY order_date) AS running_total  
FROM orders;
```

21.6 Find Customers With No Orders

```
SELECT c.customer_name  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

21.7 Second Highest Value Without LIMIT

```
SELECT MAX(amount) AS second_highest  
FROM orders  
WHERE amount < (SELECT MAX(amount) FROM orders);
```

Note: Ye trick tab useful hai jab interviewer specifically LIMIT/OFFSET use karne se mana kare.

21.8 Find Consecutive Days with Orders

English: This type of question checks whether you can combine window functions with date arithmetic — a common pattern for streak or consecutive-activity questions.

Hinglish: Ye type ka question check karta hai ki tum window functions ko date arithmetic ke saath combine kar sakte ho ya nahi — streak ya consecutive-activity wale questions ka common pattern hai.

```
SELECT customer_id, order_date,  
DATEDIFF(order_date, LAG(order_date) OVER (PARTITION BY customer_id ORDER  
BY order_date)) AS gap_days  
FROM orders;
```

Note: Agar gap_days = 1 hai, toh wo consecutive din ka order hai. LAG() previous row ki value fetch karta hai.

— End of Notes —